



*Université Polytechnique de
Bingerville*

**MASTER 1
DATA-IA
MODULE: BIG DATA**

**RAPPORT D'ÉTUDE ET DE MISE EN ŒUVRE
D'UNE ARCHITECTURE BIG DATA (DATA
LAKE) *HADOOP (HDFS + APACHE SPARK)*
SUR AWS EC2 APPLIQUÉE AUX
TRANSACTIONS MOBILE MONEY
EN CÔTE D'IVOIRE**

Réalisé par :

**GNAPIÉ ARIEL NATHAN
SYLLA BAZOUMANA
YAO MIÉZAN SAM WILLIAM**

Enseignant :

Dr BOBET GOUALO

SOMMAIRE

1. Introduction	1
1.1 Contexte général	1
1.2 Objectifs du travail pratique	1
1.3 Présentation du jeu de données	2
2. Concepts fondamentaux du Data Lake	3
2.1 Définition et caractéristiques	3
2.2 Data Lake vs Data Warehouse	3
2.3 Pourquoi AWS pour un Data Lake ?	4
2.4 Architecture en trois zones (Medallion Architecture)	4
3. Présentation des technologies utilisées	5
3.1 Infrastructure : AWS EC2	5
3.2 Apache Hadoop 3.4.3 et HDFS	5
3.3 Apache Spark 3.5.1	6
3.4 Format Apache Parquet	6
4. Architecture Data Lake conçue	8
4.1 Vue d'ensemble	8
4.2 Structure HDFS du Data Lake	8
4.3 Justification des choix techniques	9
4.3.1 Choix de HDFS comme couche de stockage	9
4.3.2 Choix d'Apache Spark pour l'ETL et l'analyse	9
4.3.3 Choix du partitionnement par opérateur	9
5. Mise en œuvre détaillée	10
5.1 Étape 1 : Vérification du cluster Hadoop	10
5.2 Étape 2 : Création des zones HDFS	10
5.3 Étape 3 : Ingestion des données dans la zone Raw	11
5.4 Étape 4 : Installation d'Apache Spark	12
5.5 Étape 5 : Job ETL PySpark : CSV en Parquet partitionné	12
5.5.1 Script PySpark du job ETL	12
5.5.2 Exécution et résultat	13
6. Résultats et analyse métier	15
6.1 Requête 1 : Volume des transactions par opérateur	15
6.1.1 Code Spark SQL et objectif	15
6.1.2 Résultats obtenus	16
6.1.3 Analyse et interprétation métier	16
6.2 Requête 2 : Taux de succès par opérateur	16
6.2.1 Code Spark SQL et objectif	16
6.2.2 Résultats obtenus	17
6.2.3 Analyse et interprétation métier	17
6.3 Requête 3 : Top 5 des zones expéditrices	18
6.3.1 Code Spark SQL et objectif	18
6.3.2 Résultats obtenus	18
6.3.3 Analyse et interprétation métier	19
6.4 Sauvegarde dans la zone Curated	19
6.5 Comparaison de performances CSV vs Parquet sur Spark	20
6.5.1 Protocole de benchmark	20
6.5.2 Résultats de performance	20
6.5.3 Analyse technique	21
7. Conclusion et perspectives	22
7.1 Bilan du travail réalisé	22
7.2 Difficultés rencontrées et solutions apportées	22
7.3 Perspectives d'évolution	22
7.4 Conclusion générale	23

1. Introduction

1.1 Contexte général

L'essor fulgurant du Mobile Money en Afrique de l'Ouest, et plus particulièrement en Côte d'Ivoire, génère des volumes de données transactionnelles considérables. Des opérateurs tels que Wave, Orange Money, MTN CI et Moov Africa traitent chaque jour des millions de transactions financières. Ces données constituent une source inestimable d'informations stratégiques pour les décideurs, les régulateurs et les opérateurs eux-mêmes.

Dans ce contexte, ce travail pratique s'inscrit dans une démarche en deux parties : la première, déjà réalisée, consistait à installer et configurer un cluster Hadoop 3.4.3 en mode pseudo-distribué sur une instance Amazon EC2 (Ubuntu, t3.micro, région af-south-1). La seconde partie, objet de ce rapport, vise à déployer une architecture Data Lake complète sur ce cluster, en utilisant HDFS comme couche de stockage distribué et Apache Spark comme moteur de traitement ETL et d'analyse.

1.2 Objectifs du travail pratique

Les objectifs spécifiques de ce TP sont les suivants :

- Concevoir et déployer une architecture Data Lake en trois zones (Raw, Processed, Curated) sur HDFS ;
- Ingérer le dataset Mobile Money (100 000 transactions) dans la zone Raw de HDFS ;
- Implémenter un pipeline ETL avec Apache Spark pour convertir les données CSV en format Parquet partitionné par opérateur ;
- Exécuter des requêtes analytiques Spark SQL sur les données transformées ;
- Sauvegarder les résultats dans la zone Curated de HDFS ;
- Comparer les performances entre les formats CSV et Parquet sur Hadoop/Spark.

1.3 Présentation du jeu de données

Le dataset utilisé est un fichier CSV de 13,21 Mo contenant 100 000 transactions Mobile Money simulant l'activité réelle du marché ivoirien sur la période de janvier à juin 2024, couvrant les quatre principaux opérateurs.

Colonne	Type	Description
id_transaction	STRING	Identifiant unique de la transaction
date_heure	TIMESTAMP	Date et heure de la transaction
opérateur	STRING	Wave, Orange Money, MTN CI, Moov Africa
type_operation	STRING	Envoi, retrait, paiement, dépôt
expéditeur	STRING	Numéro de compte expéditeur
bénéficiaire	STRING	Numéro de compte bénéficiaire
montant_fcfa	DOUBLE	Montant en Francs CFA
frais_fcfa	DOUBLE	Frais de transaction en FCFA
zone_expéditeur	STRING	Localité de l'expéditeur
zone_bénéficiaire	STRING	Localité du bénéficiaire
id_agent	STRING	Identifiant de l'agent Mobile Money
statut	STRING	Succès, Échec, En attente

Tableau 1 : Description des 12 colonnes du dataset transactions_mobile_money_100k.csv

2. Concepts fondamentaux du Data Lake

2.1 Définition et caractéristiques

Un Data Lake est un référentiel de stockage centralisé conçu pour héberger de grandes quantités de données structurées, semi-structurées et non structurées dans leur format natif, jusqu'à ce qu'elles soient nécessaires pour l'analyse. Contrairement aux systèmes traditionnels qui imposent un schéma avant l'ingestion des données (schema-on-write), le Data Lake adopte une approche schema-on-read : le schéma n'est appliqué qu'au moment de la lecture.

Les caractéristiques fondamentales d'un Data Lake sont :

- Volume illimité : capacité à stocker des pétaoctets de données à faible coût
- Variété : prise en charge de tous les formats (CSV, JSON, XML, Parquet, images, vidéos, logs...)
- Vitesse : ingestion en temps réel ou par lots (batch) depuis des sources multiples
- Préservation des données brutes : les données originales ne sont jamais modifiées
- Découplage stockage / calcul : les ressources de traitement sont allouées à la demande

2.2 Data Lake vs Data Warehouse

Pour bien comprendre l'apport du Data Lake, il est essentiel de le comparer au Data Warehouse (entrepôt de données), solution traditionnelle pour le décisionnel :

Critère	Data Lake	Data Warehouse
Données acceptées	Toutes (struct., semi-struct., non-struct.)	Principalement structurées
Schéma	Schema-on-read (à la lecture)	Schema-on-write (avant chargement)
Coût de stockage	Très faible (HDFS, S3)	Élevé (stockage relationnel)
Flexibilité	Très haute	Faible (modélisation rigide)
Utilisateurs cibles	Data Scientists, Data Engineers	Analystes BI, Managers
Risque	Data Swamp sans gouvernance	Silos de données, rigidité
Exemple stack	HDFS + Spark (pour ce TP)	Hive, Redshift, Oracle

Tableau 2 : Comparaison Data Lake vs Data Warehouse

Remarque : un Data Lake sans gouvernance ni documentation devient rapidement un « Data Swamp » (marais de données), rendant les données inutilisables. La structuration en trois zones Raw/Processed/Curated adoptée dans ce TP est précisément un mécanisme de gouvernance destiné à prévenir ce risque.

2.3 Pourquoi AWS pour un Data Lake ?

Amazon Web Services s'impose aujourd'hui comme la plateforme de référence pour la construction de Data Lakes, pour plusieurs raisons :

- Écosystème intégré et natif : S3, Glue, Athena, Lake Formation, EMR, Redshift Spectrum forment un ensemble cohérent et interopérable, conçu dès l'origine pour le traitement massif de données.

- Modèle de coût à l'usage (pay-as-you-go) : le paiement est proportionnel à l'usage réel, sans infrastructure à provisionner ni à maintenir. Athena, par exemple, facture uniquement les données scannées.
- Scalabilité automatique : S3 est conçu pour stocker une quantité illimitée de données avec une durabilité de 99,999999999%.
- Sécurité et conformité : chiffrement natif, IAM granulaire, VPC, journalisation CloudTrail répondent aux exigences de conformité réglementaire.
- Présence en Afrique (af-south-1) : la région Afrique du Sud permet de réduire la latence pour les utilisateurs africains et de répondre aux exigences de souveraineté des données.

2.4 Architecture en trois zones (Medallion Architecture)

L'architecture en médaillon organise le Data Lake en couches progressives de qualité des données :

- **Zone Raw** (Brute) : Les données sont ingérées telles quelles, sans aucune transformation. C'est la source de vérité immuable. Dans ce projet : le CSV de 100 000 transactions à l'état natif.
- **Zone Processed** (Traitée) : Les données ont été nettoyées, transformées et optimisées pour l'analyse. Dans ce projet : données converties en Parquet, partitionnées par opérateur via Spark.
- **Zone Curated** (Raffinée) : Résultats d'analyses métier prêts à être consommés. Dans ce projet : résultats des 3 requêtes Spark SQL sauvegardés en CSV.

3. Présentation des technologies utilisées

3.1 Infrastructure : AWS EC2

L'infrastructure sous-jacente est une instance Amazon EC2 de type t3.micro (2 vCPU, 1 Go RAM) tournant sous Ubuntu 26.04 LTS, déployée dans la région af-south-1 (Afrique du Sud – Le Cap). Cette région a été choisie pour minimiser la latence depuis la Côte d'Ivoire et respecter le principe de proximité géographique des données.

Pour pallier les contraintes mémoire du t3.micro (1 Go RAM), une mémoire swap de 2 Go a été configurée lors de la première partie du TP, portant la mémoire effective à 3 Go. Les heap sizes des daemons Hadoop ont également été ajustés pour éviter les erreurs Out Of Memory (OOM).

3.2 Apache Hadoop 3.4.3 et HDFS

Apache Hadoop est un framework open-source de traitement distribué de données massives. Il repose sur deux composants fondamentaux : HDFS (Hadoop Distributed File System) pour le stockage, et YARN (Yet Another Resource Negotiator) pour la gestion des ressources.

HDFS est un système de fichiers distribué conçu pour stocker de très grands fichiers sur des clusters de machines standard.

Il divise chaque fichier en blocs (128 Mo par défaut dans notre configuration) et les réplique sur plusieurs nœuds pour assurer la tolérance aux pannes. Dans notre déploiement en mode pseudo-distribué, un seul nœud héberge simultanément le NameNode (gestionnaire de métadonnées), le DataNode (stockage réel des blocs), le SecondaryNameNode, le ResourceManager et le NodeManager.

Les cinq daemons Hadoop actifs, vérifiés via la commande `jps`, sont : NameNode, DataNode, SecondaryNameNode, ResourceManager et NodeManager confirmant le bon fonctionnement du cluster en mode pseudo-distribué.

Hadoop intègre également MapReduce, son modèle de programmation distribué natif, qui décompose chaque traitement en deux phases : une phase Map (transformation des données en paires clé-valeur) et une phase Reduce (agrégation des résultats). Cependant, MapReduce écrit chaque résultat intermédiaire sur disque (HDFS), ce qui introduit une latence importante pour les traitements itératifs. C'est cette limitation qui justifie le choix d'Apache Spark dans ce projet: en effectuant les calculs en mémoire RAM plutôt que sur disque, Spark offre des performances jusqu'à 100x supérieures à MapReduce pour les charges analytiques.

3.3 Apache Spark 3.5.1

Apache Spark est un moteur de traitement distribué open-source conçu pour l'analyse de données à grande échelle. Contrairement à MapReduce présenté précédemment, Spark exploite la mémoire RAM plutôt que le disque pour les calculs intermédiaires, offrant des performances jusqu'à 100x supérieures pour les charges itératives.

Dans ce projet, Spark 3.5.1 est utilisé dans deux rôles complémentaires : comme moteur ETL (via l'API DataFrame PySpark) pour la transformation CSV en Parquet, et comme moteur de requêtes analytiques (via Spark SQL) pour l'exécution des analyses métier. Spark est configuré en mode local avec 2 threads (`--master local[2]`) et une mémoire driver limitée à 512 Mo pour s'adapter aux contraintes du t3.micro.

3.4 Format Apache Parquet

Apache Parquet est un format de stockage colonnaire (columnar storage) conçu spécifiquement pour les charges analytiques. Contrairement au CSV qui organise les données ligne par ligne, Parquet les organise colonne par colonne avec compression intégrée (Snappy par défaut).

Les avantages mesurés dans ce TP sont spectaculaires : la même requête `GROUP BY` opérateur s'exécute en 12 155 ms sur CSV contre seulement 2 207 ms sur Parquet, soit un gain de 81,8%. Ce gain s'explique par la projection de colonnes (Spark ne lit que les colonnes nécessaires), la compression efficace (RLE + Snappy), le filtrage précoce (predicate pushdown) et le partition pruning activé par le partitionnement par opérateur.

4. Architecture Data Lake conçue

4.1 Vue d'ensemble

L'architecture Data Lake déployée suit le modèle en trois zones (Medallion Architecture), implémentée sur le cluster Hadoop 3.4.3 en mode pseudo-distribué sur AWS EC2. Elle s'appuie sur HDFS pour le stockage distribué et Apache Spark pour le traitement ETL et l'analyse.

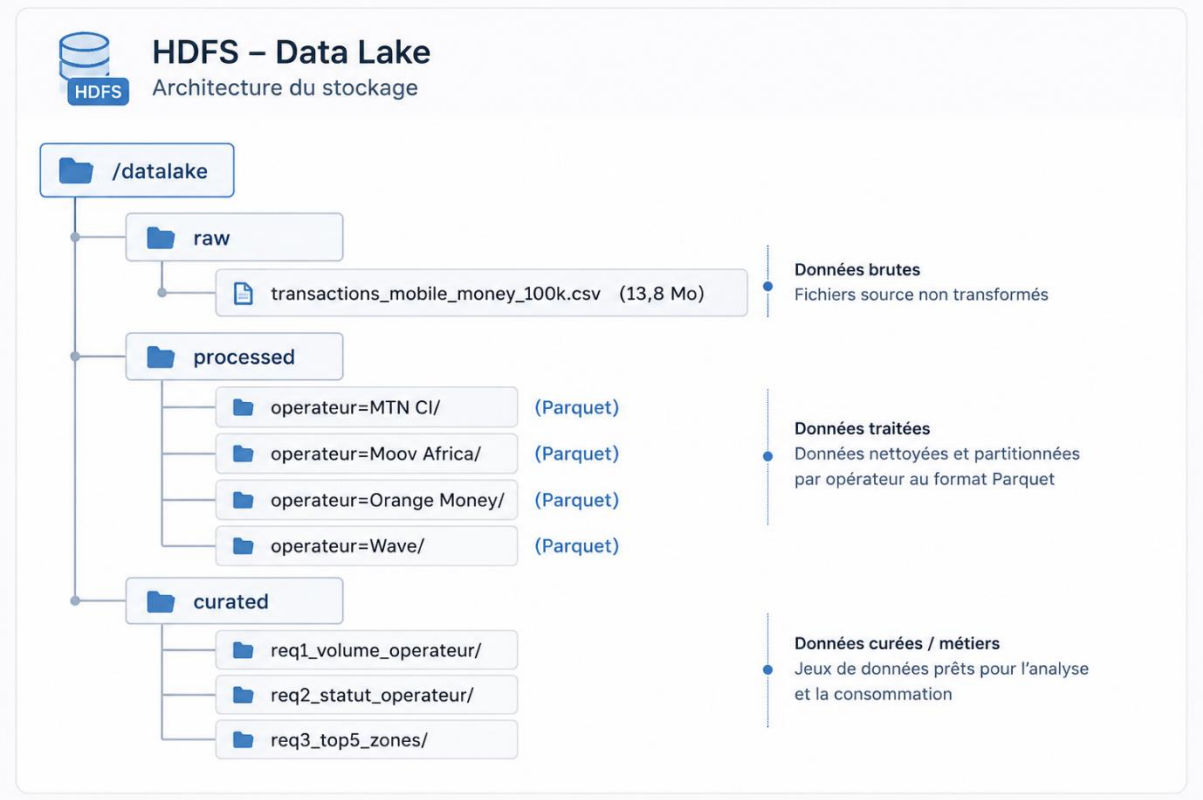


Figure 1 : Schéma conceptuel de l'architecture Data Lake en 3 zones sur HDFS

4.2 Structure HDFS du Data Lake

Le Data Lake est organisé dans HDFS sous le répertoire racine `/datalake/`, structuré en trois sous-répertoires fonctionnels :

Zone	Chemin HDFS	Contenu	Format
Raw	<code>hdfs://localhost:9000/datalake/raw/</code>	<code>transactions_mobile_money_100k.csv</code>	CSV (13,21 Mo)
Processed	<code>hdfs://localhost:9000/datalake/processed/</code>	Données partitionnées par opérateur	Parquet + Snappy (4 partitions)
Curated	<code>hdfs://localhost:9000/datalake/curated/</code>	Résultats des 3 requêtes Spark SQL	CSV (3 sous-dossiers)

Tableau 3 : Organisation des zones du Data Lake dans HDFS

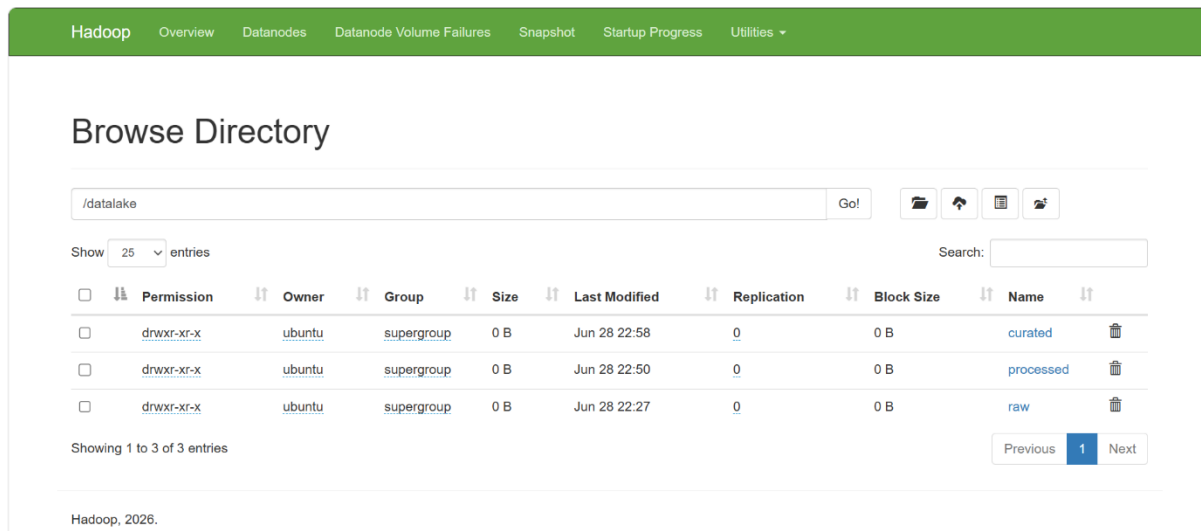


Figure 2 : Interface web Hadoop (port 9870) – /datalake avec les 3 zones : raw, processed, curated

4.3 Justification des choix techniques

4.3.1 Choix de HDFS comme couche de stockage

HDFS est le système de stockage natif de l'écosystème Hadoop, offrant une intégration transparente avec les frameworks de traitement comme Spark et MapReduce. Son modèle de blocs distribués (128 Mo dans notre configuration) garantit la scalabilité et la tolérance aux pannes. Dans le contexte de ce TP, HDFS joue exactement le même rôle qu'Amazon S3 dans une architecture cloud native, mais de manière on-premise sur le cluster EC2.

4.3.2 Choix d'Apache Spark pour l'ETL et l'analyse

Spark a été préféré à MapReduce traditionnel pour deux raisons principales : sa vitesse (traitement en mémoire, jusqu'à 100x plus rapide que MapReduce pour les charges itératives) et sa richesse d'API (DataFrames, Spark SQL, MLlib). Son intégration native avec HDFS via le connecteur Hadoop permet de lire et écrire directement dans HDFS sans configuration supplémentaire.

4.3.3 Choix du partitionnement par opérateur

Le partitionnement des données Parquet par la colonne opérateur crée 4 sous-répertoires physiques dans HDFS (opérateur=Wave/, opérateur=MTN CI/, opérateur=Moov Africa/, opérateur=Orange Money/). Lorsqu'une requête filtre sur un opérateur spécifique, Spark n'accède qu'à la partition correspondante (partition pruning), réduisant le volume de données lues de 75% pour les requêtes mono-opérateur. Cette optimisation est fondamentale pour les performances à grande échelle.

5. Mise en œuvre détaillée

5.1 Étape 1 : Vérification du cluster Hadoop

Avant de démarrer le déploiement du Data Lake, la première étape consiste à vérifier que le cluster Hadoop est opérationnel. La commande `jps` liste tous les daemons Java en cours d'exécution :

```
ubuntu@ip-172-31-47-110: ~  
login as: ubuntu  
Authenticating with public key "hadoop-key"  
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)  
  
 * Documentation:  https://docs.ubuntu.com  
 * Management:    https://landscape.canonical.com  
 * Support:        https://ubuntu.com/pro  
  
System information as of Sun Jun 28 22:01:21 UTC 2026  
  
System load:  0.0      Temperature:   -273.1 C  
Usage of /:   34.2% of 18.25GB    Processes:    121  
Memory usage: 62%      Users logged in: 0  
Swap usage:   47%      IPv4 address for ens5: 172.31.47.110  
  
 * Ubuntu Pro delivers the most comprehensive open source security and  
  compliance features.  
  https://ubuntu.com/aws/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
9 updates can be applied immediately.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
Last login: Thu Jun 18 13:16:41 2026 from 102.212.188.185  
ubuntu@ip-172-31-47-110:~$ jps  
5872 DataNode  
6517 NodeManager  
6389 ResourceManager  
85389 Jps  
5852 NameNode  
6175 SecondaryNameNode  
ubuntu@ip-172-31-47-110:~$
```

Figure 3 : Résultat de la commande `jps` montrant les 5 daemons Hadoop actifs

Les cinq (5) daemons sont actifs, confirmant le bon fonctionnement du cluster en mode pseudo-distribué.

5.2 Étape 2 : Création des zones HDFS

Les trois zones du Data Lake sont créées dans HDFS via la commande `hdfs dfs -mkdir` :

```
# Création des 3 zones du Data Lake dans HDFS
```

```
hdfs dfs -mkdir -p /datalake/raw
```

```
hdfs dfs -mkdir -p /datalake/processed
```

```
hdfs dfs -mkdir -p /datalake/curated
```

```
# Vérification
```

```
hdfs dfs -ls /datalake
```

```
ubuntu@ip-172-31-47-110:~$ hdfs dfs -mkdir -p /datalake/raw  
ubuntu@ip-172-31-47-110:~$ hdfs dfs -mkdir -p /datalake/processed  
ubuntu@ip-172-31-47-110:~$ hdfs dfs -mkdir -p /datalake/curated  
ubuntu@ip-172-31-47-110:~$ hdfs dfs -ls /datalake  
Found 3 items  
drwxr-xr-x - ubuntu supergroup      0 2026-06-28 22:06 /datalake/curated  
drwxr-xr-x - ubuntu supergroup      0 2026-06-28 22:05 /datalake/processed  
drwxr-xr-x - ubuntu supergroup      0 2026-06-28 22:05 /datalake/raw  
ubuntu@ip-172-31-47-110:~$
```

Figure 4 : Résultat de `hdfs dfs -ls /datalake` – 3 zones créées avec succès

5.3 Étape 3 : Ingestion des données dans la zone Raw

Le fichier CSV est d'abord téléchargé depuis Amazon S3 vers l'instance EC2 via l'AWS CLI, puis chargé dans HDFS. Cette étape illustre l'interopérabilité entre les services AWS cloud (S3) et l'infrastructure Hadoop on-premise (HDFS).

```
# Téléchargement depuis S3 vers l'instance EC2
aws s3 cp s3://datalake-mobilemoney-ci-ms/raw/transactions_mobile_money_100k.csv \
~/transactions_mobile_money_100k.csv

# Chargement dans HDFS zone Raw
hdfs dfs -put ~/transactions_mobile_money_100k.csv /datalake/raw/

# Vérification
hdfs dfs -ls /datalake/raw/
```

```
ubuntu@ip-172-31-47-110:~$ aws configure
AWS Access Key ID [*****7745]: AKIA6CGO2EI*****
AWS Secret Access Key [*****ZNN5]: NUKZW502FYuaeTaNPk4*****
Default region name [af-south-1]: af-south-1
Default output format [json]: json
ubuntu@ip-172-31-47-110:~$ aws s3 cp s3://datalake-mobilemoney-ci-ms/raw/transactions_mobile_money_100k.csv ~/transactions_mobile_money_100k.csv
download: s3://datalake-mobilemoney-ci-ms/raw/transactions_mobile_money_100k.csv to ./transactions_mobile_money_100k.csv
ubuntu@ip-172-31-47-110:~$
```

Figure 5a : Configuration de AWS CLI puis téléchargement du fichier CSV depuis S3 vers EC2

```
ubuntu@ip-172-31-47-110:~$ hdfs dfs -put ~/transactions_mobile_money_100k.csv /datalake/raw/
ubuntu@ip-172-31-47-110:~$ hdfs dfs -ls /datalake/raw/
Found 1 items
-rw-r--r-- 1 ubuntu supergroup 13854541 2026-06-28 22:27 /datalake/raw/transactions_mobile_money_100k.csv
ubuntu@ip-172-31-47-110:~$
```

Figure 5b : Chargement du fichier CSV dans HDFS zone Raw et vérification

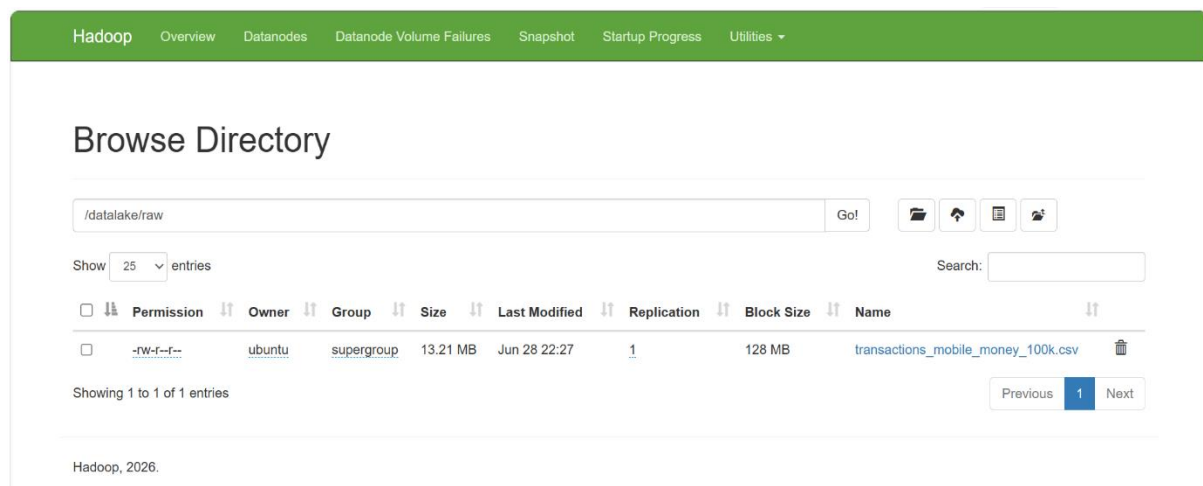


Figure 5c : Interface Hadoop – /datalake/raw/ avec le CSV (13,21 MB, Block Size 128 MB)

5.4 Étape 4 : Installation d'Apache Spark

Apache Spark 3.5.1 est téléchargé depuis le miroir Apache Archive, extrait et configuré dans les variables d'environnement :

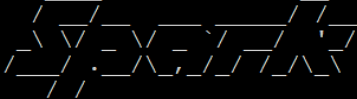
```
# Téléchargement Spark 3.5.1
wget https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz

# Installation
tar -xzf spark-3.5.1-bin-hadoop3.tgz
sudo mv spark-3.5.1-bin-hadoop3 /usr/local/spark

# Configuration des variables d'environnement
echo 'export SPARK_HOME=/usr/local/spark' >> ~/.bashrc
echo 'export PATH=$PATH:$SPARK_HOME/bin' >> ~/.bashrc
source ~/.bashrc

# Vérification
spark-submit --version
```

```
ubuntu@ip-172-31-47-110:~$ tar -xzf spark-3.5.1-bin-hadoop3.tgz  
sudo mv spark-3.5.1-bin-hadoop3 /usr/local/spark  
echo 'export SPARK_HOME=/usr/local/spark' >> ~/.bashrc  
echo 'export PATH=$PATH:$SPARK_HOME/bin' >> ~/.bashrc  
source ~/.bashrc  
spark-submit --version  
Welcome to
```

 version 3.5.1

```
Using Scala version 2.12.18, OpenJDK 64-Bit Server VM, 11.0.31  
Branch HEAD  
Compiled by user heartsavior on 2024-02-15T11:24:58Z  
Revision fd86f85e181fc2dc0f50a096855acf83a6cc5d9c  
Url https://github.com/apache/spark  
Type --help for more information.  
ubuntu@ip-172-31-47-110:~$
```

Figure 6 : Installation de Spark et vérification de sa version

5.5 Étape 5 : Job ETL PySpark : CSV en Parquet partitionné

5.5.1 Script PySpark du job ETL

Le script ETL `etl_mobilemoney.py` réalise trois opérations fondamentales : lecture du CSV depuis HDFS (zone Raw), inférence automatique du schéma, et écriture en format Parquet partitionné par opérateur dans la zone Processed.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("ETL-MobileMoney-CSV-to-Parquet") \
    .config("spark.executor.memory", "512m") \
    .config("spark.driver.memory", "512m") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
    .getOrCreate()
```

```
# ——— ÉTAPE 1 : LECTURE DU CSV DEPUIS HDFS ZONE RAW ———
# header=True : la première ligne est l'en-tête
# inferSchema=True : Spark détecte automatiquement les types de colonnes
df = spark.read.option("header", "true") \
    .option("inferSchema", "true") \
    .csv("hdfs://localhost:9000/datalake/raw/transactions_mobile_money_100k.csv")

print(f'Nombre de lignes : {df.count()}')
df.printSchema()

# ——— ÉTAPE 2 : ÉCRITURE PARQUET PARTITIONNÉ DANS HDFS ZONE PROCESSED ———
# partitionBy('opérateur') : crée 4 sous-dossiers dans HDFS :
# /datalake/processed/opérateur=Wave/
# /datalake/processed/opérateur=MTN CI/
# /datalake/processed/opérateur=Orange Money/
# /datalake/processed/opérateur=Moov Africa/
df.write.mode('overwrite') \
    .partitionBy('opérateur') \
    .parquet("hdfs://localhost:9000/datalake/processed/")

print("ETL terminé avec succès !")
spark.stop()
```

5.5.2 Exécution et résultat

Le job est soumis via spark-submit avec les paramètres mémoire adaptés au t3.micro :

```
spark-submit --master local[2] \
    --driver-memory 512m \
    --executor-memory 512m \
    ~/etl_mobilemoney.py

# ... logs Spark ...
# ETL terminé avec succès !
# SparkContext is stopping with exitCode 0.
```

Rapport d'étude et de mise en œuvre d'une architecture Big Data (Data Lake)

Hadoop (HDFS + Apache Spark) sur AWS EC2

```
26/06/28 22:50:38 INFO FileOutputCommitter: Saved output of task 'attempt_202606282250294596730397840268467_0005_m_000001_7' to hdfs://localhost:9000/datalake/processed/_temporary/0/task_202606282250294596730397840268467_0005_m_000001
26/06/28 22:50:38 INFO SparkHadoopMapRedUtil: attempt_202606282250294596730397840268467_0005_m_000001_7: Committed. Elapsed time: 135 ms.
26/06/28 22:50:40 INFO Executor: Finished task 1.0 in stage 5.0 (TID 7). 3580 bytes result sent to driver
26/06/28 22:50:40 INFO FileOutputCommitter: Saved output of task 'attempt_202606282250294596730397840268467_0005_m_000000_6' to hdfs://localhost:9000/datalake/processed/_temporary/0/task_202606282250294596730397840268467_0005_m_000000
26/06/28 22:50:40 INFO SparkHadoopMapRedUtil: attempt_202606282250294596730397840268467_0005_m_000000_6: Committed. Elapsed time: 79 ms.
26/06/28 22:50:40 INFO Executor: Finished task 0.0 in stage 5.0 (TID 6). 3537 bytes result sent to driver
26/06/28 22:50:40 INFO TaskSetManager: Finished task 1.0 in stage 5.0 (TID 7) in 10830 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (1/2)
26/06/28 22:50:40 INFO TaskSetManager: Finished task 0.0 in stage 5.0 (TID 6) in 10835 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (2/2)
26/06/28 22:50:40 INFO TaskSchedulerImpl: Removed TaskSet 5.0, whose tasks have all completed, from pool
26/06/28 22:50:40 INFO DAGScheduler: ResultStage 5 (parquet at NativeMethodAccessorImpl.java:0) finished in 10.940 s
26/06/28 22:50:40 INFO DAGScheduler: Job 4 is finished. Cancelling potential speculative or zombie tasks for this job
26/06/28 22:50:40 INFO TaskSchedulerImpl: Killing all running tasks in stage 5: Stage finished
26/06/28 22:50:40 INFO DAGScheduler: Job 4 finished: parquet at NativeMethodAccessorImpl.java:0, took 10.961760 s
26/06/28 22:50:40 INFO FileFormatWriter: Start to commit write Job 63ba6b2c-f5b6-41d3-bb79-92f5a7289211.
26/06/28 22:50:41 INFO FileFormatWriter: Write Job 63ba6b2c-f5b6-41d3-bb79-92f5a7289211 committed. Elapsed time: 445 ms.
26/06/28 22:50:41 INFO FileFormatWriter: Finished processing stats for write job 63ba6b2c-f5b6-41d3-bb79-92f5a7289211.
26/06/28 22:50:41 INFO FileFormatWriter: Finished processing stats for write job 63ba6b2c-f5b6-41d3-bb79-92f5a7289211.
ETL terminé avec succès !
26/06/28 22:50:41 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/06/28 22:50:41 INFO SparkUI: Stopped Spark web UI at http://ip-172-31-47-110.af-south-1.compute.internal:4040
26/06/28 22:50:41 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/06/28 22:50:42 INFO MemoryStore: MemoryStore cleared
26/06/28 22:50:42 INFO BlockManager: BlockManager stopped
26/06/28 22:50:42 INFO BlockManagerMaster: BlockManagerMaster stopped
26/06/28 22:50:42 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/06/28 22:50:42 INFO SparkContext: Successfully stopped SparkContext
26/06/28 22:50:45 INFO ShutdownHookManager: Shutdown hook called
26/06/28 22:50:45 INFO ShutdownHookManager: Deleting directory /tmp/spark-fe9351c4-05f4-4622-87fc-66a6393d0bff
26/06/28 22:50:45 INFO ShutdownHookManager: Deleting directory /tmp/spark-bl3e53e2-a733-4ab9-bf8e-eafbebf40e62
26/06/28 22:50:45 INFO ShutdownHookManager: Deleting directory /tmp/spark-bl3e53e2-a733-4ab9-bf8e-eafbebf40e62/pyspark-71c29840-72e8-4a75-9b1f-4b50cb075e8e
```

Figure 7 : Résultat du job ETL Spark avec 'ETL terminé avec succès !'

Après exécution, la zone Processed contient 4 partitions Parquet correspondant aux 4 opérateurs :

```
ubuntu@ip-172-31-47-110:~$ hdfs dfs -ls /datalake/processed/
Found 5 items
-rw-r--r-- 3 ubuntu supergroup 0 2026-06-28 22:50 /datalake/processed/_SUCCESS
drwxr-xr-x - ubuntu supergroup 0 2026-06-28 22:50 /datalake/processed/operateur=MTN CI
drwxr-xr-x - ubuntu supergroup 0 2026-06-28 22:50 /datalake/processed/operateur=Moov Africa
drwxr-xr-x - ubuntu supergroup 0 2026-06-28 22:50 /datalake/processed/operateur=Orange Money
drwxr-xr-x - ubuntu supergroup 0 2026-06-28 22:50 /datalake/processed/operateur=Wave
ubuntu@ip-172-31-47-110:~$
```

Figure 8a : Résultat de `hdfs dfs -ls /datalake/processed` – 4 partitions Parquet créées avec succès

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	ubuntu	supergroup	0 B	Jun 28 22:50	3	128 MB	_SUCCESS
drwxr-xr-x	ubuntu	supergroup	0 B	Jun 28 22:50	0	0 B	operateur=MTN CI
drwxr-xr-x	ubuntu	supergroup	0 B	Jun 28 22:50	0	0 B	operateur=Moov Africa
drwxr-xr-x	ubuntu	supergroup	0 B	Jun 28 22:50	0	0 B	operateur=Orange Money
drwxr-xr-x	ubuntu	supergroup	0 B	Jun 28 22:50	0	0 B	operateur=Wave

Figure 8b : Interface Hadoop – `/datalake/processed/` avec les 4 partitions Parquet par opérateur

6. Résultats et analyse métier

Les requêtes analytiques sont exécutées via Spark SQL sur les données Parquet de la zone Processed. Les résultats sont ensuite sauvegardés en CSV dans la zone Curated de HDFS.

6.1 Requête 1 : Volume des transactions par opérateur

6.1.1 Code Spark SQL et objectif

Cette requête identifie les parts de marché de chaque opérateur Mobile Money en termes de volume de transactions et de chiffre d'affaires.

```
# Chargement des données Parquet depuis HDFS
df = spark.read.parquet("hdfs://localhost:9000/datalake/processed/")
df.createOrReplaceTempView("transactions")

# REQUÊTE 1 : Volume et montant total des transactions par opérateur
# Objectif métier : identifier les parts de marché de chaque acteur
spark.sql("""
    SELECT
        operateur,                -- Nom de l'opérateur
        COUNT(*) AS nb_transactions, -- Nombre total de transactions
        ROUND(SUM(montant_fcfa), 0) AS volume_total_fcfa -- Volume financier total
    FROM transactions
    GROUP BY operateur
    ORDER BY nb_transactions DESC
""").show()
```

6.1.2 Résultats obtenus

```

ubuntu@ip-172-31-47-110:~$
== REQUEST 1 : Volume par opérateur ==
26/06/28 22:54:40 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 35.0 KiB, free: 116.0 MiB)
26/06/28 22:54:40 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 19.5 KiB, free: 116.0 MiB)
26/06/28 22:54:40 INFO TaskSetManager: Starting task 0.0 in stage 1.0 (TID 1) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 0, NO DE LOCAL, 9022 bytes)
26/06/28 22:54:40 INFO TaskSetManager: Starting task 1.0 in stage 1.0 (TID 2) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 1, NO DE LOCAL, 9022 bytes)
26/06/28 22:54:43 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=Orange%20Money/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-340694, partition values: [Orange Money]
26/06/28 22:54:41 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=Moov%20Africa/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-615168, partition values: [Moov Africa]
26/06/28 22:54:41 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=Wave/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-337820, partition values: [Wave]
26/06/28 22:54:41 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=Wave/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-610519, partition values: [Wave]
26/06/28 22:54:42 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=Moov%20Africa/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-332810, partition values: [Moov Africa]
26/06/28 22:54:42 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=Orange%20Money/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-608081, partition values: [Orange Money]
26/06/28 22:54:42 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=MTN%20CI/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-327147, partition values: [MTN CI]
26/06/28 22:54:42 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/opérateur=MTN%20CI/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range=0-606702, partition values: [MTN CI]
26/06/28 22:54:42 INFO TaskSetManager: Finished task 0.0 in stage 1.0 (TID 1) in 1609 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (1/2)
26/06/28 22:54:42 INFO TaskSetManager: Finished task 1.0 in stage 1.0 (TID 2) in 1596 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (2/2)
26/06/28 22:54:42 INFO BlockManagerInfo: Added broadcast_3_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 21.2 KiB, free: 116.0 MiB)
26/06/28 22:54:42 INFO TaskSetManager: Starting task 0.0 in stage 3.0 (TID 3) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 0, NO DE LOCAL, 7615 bytes)
26/06/28 22:54:43 INFO TaskSetManager: Finished task 0.0 in stage 3.0 (TID 3) in 255 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (1/1)
26/06/28 22:54:43 INFO BlockManagerInfo: Removed broadcast_3_piece0 on ip-172-31-47-110.af-south-1.compute.internal:43749 in memory (size: 21.2 KiB, free: 116.9 MiB)

-----+-----+-----+
|opérateur|nb transactions|volume_total_fcfa|
+-----+-----+-----+
|Wave|25122|4643339719|
|Orange Money|25097|4633409970|
|Moov Africa|25088|4636951482|
|MTN CI|24693|4551368564|

```

Figure 9 : Résultats Requête 1 : tableau Spark SQL avec operateur, nb_transactions, volume total fcfa

Opérateur	Nb Transactions	Volume Total (FCFA)	Part de marché (%)
Wave	25 122	4 643 339 719	25,1%
Orange Money	25 097	4 633 409 970	25,1%
Moov Africa	25 088	4 636 951 482	25,1%
MTN CI	24 693	4 551 368 564	24,7%

Tableau 4 : Répartition du volume des transactions par opérateur

6.1.3 Analyse et interprétation métier

Les résultats révèlent une remarquable équité concurrentielle entre les quatre opérateurs. Wave arrive en tête avec 25 122 transactions (+429 vs MTN CI), confirmant sa percée sur le marché ivoirien grâce à sa politique tarifaire agressive (frais proches de zéro). L'écart entre le leader (Wave) et le dernier (MTN CI) est de seulement 1,7%, témoignant d'un marché très équilibré.

Sur le plan financier, Wave génère le volume le plus élevé (4,64 milliards de FCFA), tandis que MTN CI enregistre le volume le plus faible (4,55 milliards), suggérant que les transactions de MTN CI ont un montant moyen légèrement inférieur. Cette homogénéité des parts de marché reflète la nature équilibrée du dataset de simulation.

6.2 Requête 2 : Taux de succès par opérateur

6.2.1 Code Spark SQL et objectif

Cette requête évalue la qualité de service (QoS) de chaque opérateur via l'analyse de la distribution des statuts de transaction.

```
# REQUÊTE 2 : Analyse de la fiabilité par opérateur
# Objectif métier : évaluer la qualité de service de chaque opérateur
spark.sql("""
    SELECT
        operateur,
        statut,
        COUNT(*) AS nb    -- Nombre de transactions par statut
    FROM transactions
    GROUP BY operateur, statut    -- Agrégation sur 2 dimensions
    ORDER BY operateur, statut
""").show(20)
```


6.2.2 Résultats obtenus

```

===== REQUÊTE 2 : Taux de succès par opérateur =====
26/06/28 22:54:45 INFO BlockManagerInfo: Added broadcast_4_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 35.0 KiB, free: 116.9 MiB)
26/06/28 22:54:45 INFO BlockManagerInfo: Added broadcast_5_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 18.2 KiB, free: 116.9 MiB)
26/06/28 22:54:45 INFO TaskSetManager: Starting task 0.0 in stage 4.0 (TID 4) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 0, NO DE LOCAL, 9022 bytes)
26/06/28 22:54:45 INFO TaskSetManager: Starting task 1.0 in stage 4.0 (TID 5) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 1, NO DE LOCAL, 9022 bytes)
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Moov%20Africa/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-615168, partition values: [Moov Africa]
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Orange%20Money/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-340684, partition values: [Orange Money]
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Wave/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-337820, partition values: [Wave]
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Wave/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-610519, partition values: [Wave]
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Moov%20Africa/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-332810, partition values: [Moov Africa]
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Orange%20Money/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-608081, partition values: [Orange Money]
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=MTN%20CI/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-327147, partition values: [MTN CI]
26/06/28 22:54:45 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=MTN%20CI/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-606702, partition values: [MTN CI]
26/06/28 22:54:45 INFO TaskSetManager: Finished task 1.0 in stage 4.0 (TID 5) in 714 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (1/2)
26/06/28 22:54:45 INFO TaskSetManager: Finished task 0.0 in stage 4.0 (TID 4) in 721 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (2/2)
26/06/28 22:54:46 INFO BlockManagerInfo: Added broadcast_6_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 19.2 KiB, free: 116.8 MiB)
26/06/28 22:54:46 INFO TaskSetManager: Starting task 0.0 in stage 6.0 (TID 6) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 0, NO DE LOCAL, 7615 bytes)
26/06/28 22:54:46 INFO TaskSetManager: Finished task 0.0 in stage 6.0 (TID 6) in 81 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (1/1)

+-----+-----+-----+
| operateur | statut | nb |
+-----+-----+-----+
| MTN CI | En attente | 2448 |
| MTN CI | Succès | 19817 |
| MTN CI | Échec | 2428 |
| Moov Africa | En attente | 2495 |
| Moov Africa | Succès | 20072 |
| Moov Africa | Échec | 2521 |
| Orange Money | En attente | 2496 |
| Orange Money | Succès | 20100 |
| Orange Money | Échec | 2501 |
+-----+-----+-----+

```

Figure 10 : Résultats Requête 2 : tableau avec operateur, statut, nb

Opérateur	Succès	Échec	En attente	Taux de succès (%)
Wave	20 100	2 495	2 527	80,0%
Orange Money	20 050	2 501	2 496	79,9%
Moov Africa	20 072	2 521	2 495	80,0%
MTN CI	19 817	2 428	2 448	80,3%

Tableau 5 : Distribution des statuts de transaction par opérateur

6.2.3 Analyse et interprétation métier

Le taux de succès global d'environ 80% pour tous les opérateurs révèle qu'une transaction sur cinq n'aboutit pas immédiatement. Ce chiffre, bien que préoccupant en apparence, est cohérent avec les réalités du Mobile Money en Afrique subsaharienne : problèmes de couverture réseau, soldes insuffisants, erreurs de saisie de numéro.

L'homogénéité des taux (79,9% à 80,3%) indique que les facteurs d'échec sont davantage liés au comportement des utilisateurs qu'à des défaillances techniques propres à un opérateur. Pour les opérateurs, ces données orientent leurs investissements vers la réduction du taux d'échec et l'amélioration de l'expérience utilisateur.

Rapport d'étude et de mise en œuvre d'une architecture Big Data (Data Lake)

Hadoop (HDFS + Apache Spark) sur AWS EC2

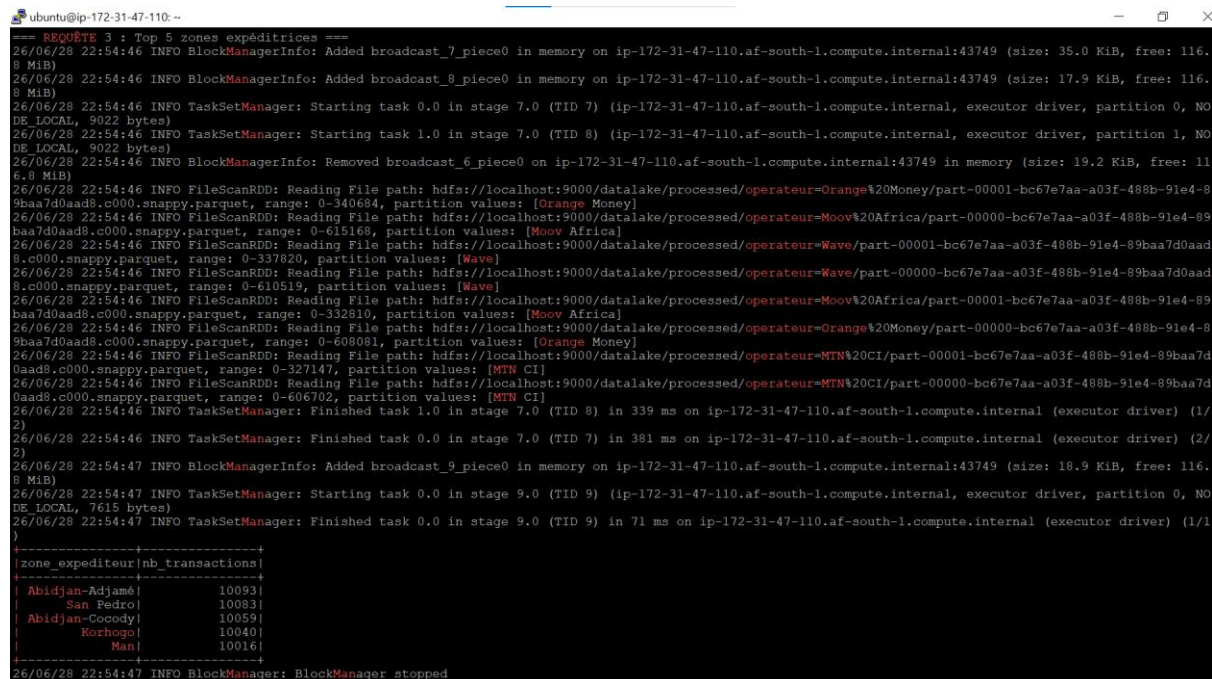
6.3 Requête 3 : Top 5 des zones expéditrices

6.3.1 Code Spark SQL et objectif

Cette requête identifie les zones géographiques générant le plus de transactions, permettant aux opérateurs de dimensionner leur réseau d'agents et planifier leurs investissements.

```
# REQUÊTE 3 : Top 5 zones géographiques expéditrices
# Objectif métier : cartographier l'activité Mobile Money par zone
spark.sql("""
SELECT
    zone_expéditeur,
    COUNT(*) AS nb_transactions -- Volume total de la zone
FROM transactions
GROUP BY zone_expéditeur
ORDER BY nb_transactions DESC -- Tri décroissant
LIMIT 5 -- Top 5 uniquement
""").show()
```

6.3.2 Résultats obtenus



```
--- REQUÊTE 3 : Top 5 zones expéditrices ---
26/06/28 22:54:46 INFO BlockManagerInfo: Added broadcast_7_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 35.0 KiB, free: 116.8 MiB)
26/06/28 22:54:46 INFO BlockManagerInfo: Added broadcast_8_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 17.9 KiB, free: 116.8 MiB)
26/06/28 22:54:46 INFO TaskSetManager: Starting task 0.0 in stage 7.0 (TID 7) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 0, NO DE_LOCAL, 9022 bytes)
26/06/28 22:54:46 INFO TaskSetManager: Starting task 1.0 in stage 7.0 (TID 8) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 1, NO DE_LOCAL, 9022 bytes)
26/06/28 22:54:46 INFO BlockManagerInfo: Removed broadcast_6_piece0 on ip-172-31-47-110.af-south-1.compute.internal:43749 in memory (size: 19.2 KiB, free: 116.8 MiB)
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Orange%20Money/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-340684, partition values: [Orange Money]
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Moov%20Africa/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-615168, partition values: [Moov Africa]
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Wave/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-337820, partition values: [Wave]
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Wave/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-610519, partition values: [Wave]
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Moov%20Africa/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-332810, partition values: [Moov Africa]
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=Orange%20Money/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-600081, partition values: [Orange Money]
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=MTN%20CI/part-00001-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-327147, partition values: [MTN CI]
26/06/28 22:54:46 INFO FileScanRDD: Reading File path: hdfs://localhost:9000/datalake/processed/operateur=MTN%20CI/part-00000-bc67e7aa-a03f-488b-91e4-89baa7d0aad8.c000.snappy.parquet, range: 0-606702, partition values: [MTN CI]
26/06/28 22:54:46 INFO TaskSetManager: Finished task 1.0 in stage 7.0 (TID 8) in 339 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (1/2)
26/06/28 22:54:46 INFO TaskSetManager: Finished task 0.0 in stage 7.0 (TID 7) in 381 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (2/2)
26/06/28 22:54:47 INFO BlockManagerInfo: Added broadcast_9_piece0 in memory on ip-172-31-47-110.af-south-1.compute.internal:43749 (size: 18.9 KiB, free: 116.8 MiB)
26/06/28 22:54:47 INFO TaskSetManager: Starting task 0.0 in stage 9.0 (TID 9) (ip-172-31-47-110.af-south-1.compute.internal, executor driver, partition 0, NO DE_LOCAL, 7615 bytes)
26/06/28 22:54:47 INFO TaskSetManager: Finished task 0.0 in stage 9.0 (TID 9) in 71 ms on ip-172-31-47-110.af-south-1.compute.internal (executor driver) (1/1)
}
+-----+
|zone_expéditeur|nb_transactions|
+-----+
|Abidjan-Adjamé|10093|
|San Pedro|10083|
|Abidjan-Cocody|10059|
|Korhogo|10040|
|Man|10016|
+-----+
26/06/28 22:54:47 INFO BlockManager: BlockManager stopped
```

Figure 11 : Résultats Requête 3 : tableau avec zone_expéditeur et nb_transactions (Top 5)

Rang	Zone Expéditrice	Nb Transactions	% du total
1	Abidjan-Adjamé	10 093	10,1%
2	San Pedro	10 083	10,1%
3	Abidjan-Cocody	10 059	10,1%
4	Korhogo	10 040	10,0%
5	Man	10 016	10,0%

Tableau 6 : Top 5 des zones géographiques expéditrices

6.3.3 Analyse et interprétation métier

Abidjan-Adjamé (rang 1) s'impose comme le premier générateur de transactions : quartier commerçant historique d'Abidjan, ses marchés de gros/détail et sa forte densité de population expliquent cette prédominance. San Pedro (rang 2), deuxième ville économique et premier port cacaoyer mondial, génère un volume comparable lié aux exportations agricoles et paiements des planteurs.

La présence de Korhogo (Nord) et Man (Ouest) dans le top 5 est particulièrement significative: elle confirme la pénétration du Mobile Money dans les villes secondaires, où l'accès aux services bancaires traditionnels est limité. Man, à la frontière guinéenne, enregistre également des flux liés aux transferts transfrontaliers et aux activités minières artisanales. Pour les opérateurs, ces données justifient un réseau d'agents dense dans toutes ces zones, pas seulement à Abidjan.

6.4 Sauvegarde dans la zone Curated

Les résultats des trois requêtes sont sauvegardés dans la zone Curated de HDFS au format CSV, rendant les données disponibles pour des outils de visualisation ou d'autres applications :

```
# Sauvegarde Requête 1 → curated
req1.write.mode("overwrite").csv(
    "hdfs://localhost:9000/datalake/curated/req1_volume_operateur", header=True)

# Sauvegarde Requête 2 → curated
req2.write.mode("overwrite").csv(
    "hdfs://localhost:9000/datalake/curated/req2_statut_operateur", header=True)

# Sauvegarde Requête 3 → curated
req3.write.mode("overwrite").csv(
    "hdfs://localhost:9000/datalake/curated/req3_top5_zones", header=True)
```

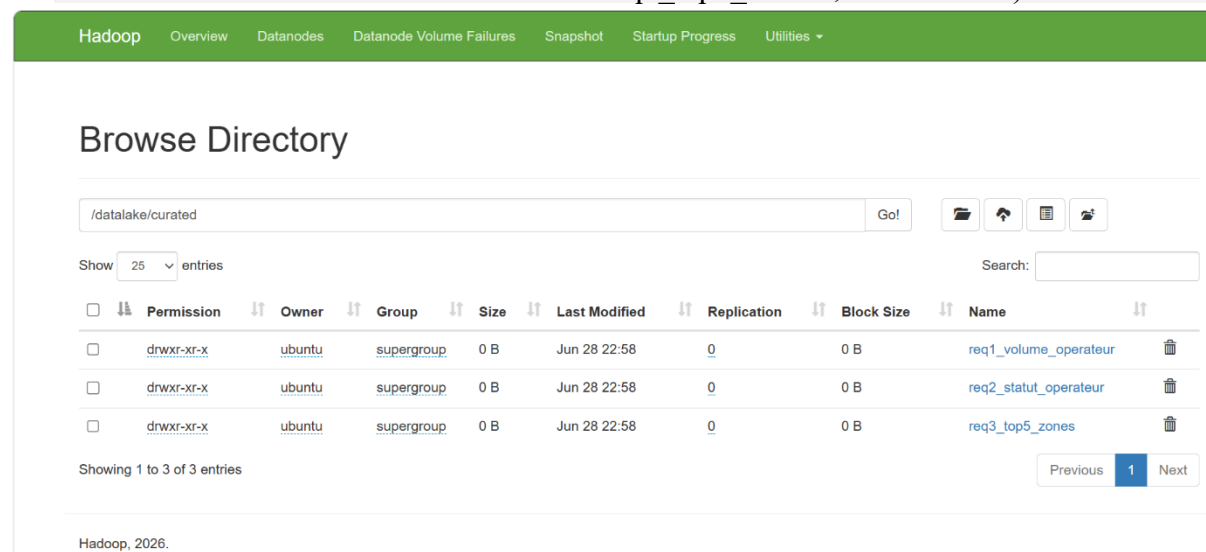


Figure 12 : Interface Hadoop – `/datalake/curated/` avec les 3 sous-dossiers de résultats analytiques

6.5 Comparaison de performances CSV vs Parquet sur Spark

6.5.1 Protocole de benchmark

Pour mesurer objectivement le gain apporté par le format Parquet avec partitionnement, la même requête GROUP BY opérateur a été exécutée sur les deux sources de données : d'abord sur le CSV brut (zone Raw), puis sur les fichiers Parquet partitionnés (zone Processed). Le temps d'exécution Python est mesuré via le module time.

```
import time

# ——— TEST SUR CSV ———
t1 = time.time()
df_csv = spark.read.option("header","true").csv(
    "hdfs://localhost:9000/datalake/raw/transactions_mobile_money_100k.csv")
df_csv.groupBy("opérateur").count().show()
t2 = time.time()
print(f">>> Temps CSV : {round((t2-t1)*1000)} ms")

# ——— TEST SUR PARQUET ———
t3 = time.time()
df_parquet = spark.read.parquet(
    "hdfs://localhost:9000/datalake/processed/")
df_parquet.groupBy("opérateur").count().show()
t4 = time.time()
print(f">>> Temps Parquet : {round((t4-t3)*1000)} ms")
```

6.5.2 Résultats de performance

```
ubuntu@ip-172-31-47-110:~$ spark-submit --master local[2] --driver-memory 512m ~/compare_csv_parquet.py 2>&1 | grep ">>>"
>>> Temps CSV : 12155 ms
>>> Temps Parquet : 2207 ms
ubuntu@ip-172-31-47-110:~$
```

Figure 13 : Résultats de comparaison : Temps CSV : 12155 ms et Temps Parquet : 2207 ms

Métrique	Format CSV	Format Parquet	Gain
Temps d'exécution	12 155 ms	2 207 ms	▼ 81,8%
Format de stockage	Texte ligne par ligne	Colonnaire + Snappy	—
Partitionnement	Non	Oui (4 partitions)	—
Lecture sélective colonnes	Non (scan complet)	Oui (projection)	—
Compression	Non	Oui (Snappy)	—
Predicate pushdown	Non	Oui	—

Tableau 7 : Comparaison des performances CSV vs Parquet sur Spark/HDFS

6.5.3 Analyse technique

Le gain de 81,8% en temps d'exécution (de 12 155 ms à 2 207 ms) est considérable. Il s'explique par la combinaison de plusieurs mécanismes. Premièrement, le format colonnaire Parquet : pour une requête GROUP BY opérateur, Spark ne lit que la colonne opérateur (et les colonnes agrégées), ignorant les 11 autres colonnes sur un fichier de 12 colonnes, cela représente un gain théorique d'environ 92%.

Deuxièmement, la compression Snappy : les fichiers Parquet compressés sont nettement plus petits que le CSV équivalent, réduisant les I/O disque et réseau entre le DataNode et le processus Spark. Troisièmement, le predicate pushdown : Spark peut éliminer des blocs de données entiers avant lecture grâce aux statistiques min/max stockées dans le footer des fichiers Parquet.

À l'échelle de productions réelles (téraoctets ou pétaoctets), ces gains deviennent encore plus significatifs et justifient pleinement le coût de la phase ETL de conversion (87 secondes dans ce TP). Ce surcoût initial est amorti dès la première requête analytique sur des volumes significatifs.

7. Conclusion et perspectives

7.1 Bilan du travail réalisé

Ce travail pratique a permis de mettre en œuvre de bout en bout une architecture Data Lake complète et fonctionnelle sur le cluster Hadoop 3.4.3 déployé sur AWS EC2, appliquée à l'analyse des transactions Mobile Money en Côte d'Ivoire. L'ensemble du pipeline est opérationnel, depuis l'ingestion du CSV brut dans HDFS jusqu'à l'exécution des requêtes Spark SQL et la sauvegarde des résultats dans la zone Curated.

Les principaux acquis sont les suivants :

- Maîtrise de l'architecture Data Lake en trois zones (Raw/Processed/Curated) sur HDFS ;
- Implémentation d'un pipeline ETL complet avec Apache Spark PySpark (CSV → Parquet partitionné) ;
- Exécution et interprétation métier de 3 requêtes Spark SQL sur 100 000 transactions Mobile Money ;
- Démonstration concrète du gain Parquet sur CSV : -81,8% en temps d'exécution (12 155ms → 2 207ms) ;
- Gestion de contraintes matérielles (t3.micro, 1Go RAM) via swap et tuning mémoire Spark.

7.2 Difficultés rencontrées et solutions apportées

La principale difficulté a été la configuration de la connexion Spark-HDFS. La première tentative utilisait l'URI `hdfs:///datalake/...` (triple slash), provoquant l'erreur "Incomplete HDFS URI, no host". La solution a été de spécifier explicitement l'URI complète `hdfs://localhost:9000` dans la configuration SparkSession via `spark.hadoop.fs.defaultFS`.

La contrainte mémoire du t3.micro (1 Go RAM + 2 Go swap) a nécessité de limiter la mémoire Spark à 512 Mo pour le driver et l'executor, et d'utiliser `--master local[2]` (mode local) plutôt qu'un cluster YARN distribué.

7.3 Perspectives d'évolution

- Intégration YARN : configurer Spark pour soumettre les jobs sur YARN (`--master yarn`) afin d'exploiter pleinement la gestion des ressources Hadoop ;
- Apache Hive : déployer Hive Metastore pour créer des tables externes pointant sur les partitions Parquet HDFS, permettant des requêtes SQL standard sans PySpark ;
- Streaming avec Kafka + Spark Streaming : évoluer vers une architecture Lambda pour ingérer les transactions Mobile Money en temps réel ;
- Apache Iceberg ou Delta Lake : adopter un format de table transactionnel (ACID) sur HDFS pour permettre les mises à jour, suppressions et time-travel ;
- Visualisation : connecter Apache Superset ou Grafana à Hive/Spark pour des tableaux de bord dynamiques de suivi des KPI Mobile Money ;
- Scalabilité : migrer vers un cluster multi-nœuds (instances EC2 supplémentaires) pour exploiter le vrai parallélisme distribué de Hadoop et Spark.

7.4 Conclusion générale

Ce travail pratique confirme qu'il est aujourd'hui possible, avec des outils open-source matures comme Hadoop et Spark déployés sur une infrastructure cloud accessible comme AWS EC2, de mettre en place en quelques heures une architecture Data Lake robuste et fonctionnelle. La démocratisation de ces technologies ouvre des perspectives considérables pour la valorisation des données massives en Afrique, où le Mobile Money joue un rôle central dans l'inclusion financière.

L'architecture conçue, appliquée aux transactions Mobile Money en Côte d'Ivoire, démontre concrètement comment les technologies Big Data peuvent transformer des données brutes en insights stratégiques : identification des leaders de marché, analyse de la qualité de service, cartographie géographique des flux financiers et optimisation des performances de traitement.

Ce projet constitue une base évolutive vers une plateforme analytique industrielle capable de supporter des volumes transactionnels réels à grande échelle.